

Top 10 Databricks Interview Questions & Answers (2026, Scenario-Based)

1. What is the difference between Databricks SQL Warehouse, Cluster & Jobs Cluster?

Answer:

Feature	All-Purpose Cluster	Jobs Cluster	SQL Warehouse
Use Case	Dev/Exploration	Scheduled Pipelines	BI/SQL Queries
Runtime	Continuous	Created-destroyed per job	Always available
Cost	High	Optimized	Moderate
Languages	Scala/Python/SQL	Scala/Python/SQL	SQL only

Real Use Case:

Use **Jobs Clusters** for ETL batch pipelines, **Warehouse** for dashboards connected to Power BI, and **All-purpose cluster** for notebooks during development.

2. What is Autoloader and when do you use it?

Answer:

Autoloader is used to ingest streaming or incrementally arriving files (CSV, JSON, Parquet) from cloud storage like ADLS/S3.

Benefits:

- Schema inference + evolution
- Checkpointing
- Fault tolerance
- Handles millions of files efficiently

Example (Bronze Ingestion):

```
df = (  
    spark.readStream.format("cloudFiles")  
    .option("cloudFiles.format", "json")  
    .load("/mnt/raw/orders")  
)
```

)

3. Explain the Bronze, Silver, Gold (Delta Architecture) layers.

Answer:

Bronze: Raw ingestion (Autoloader, CDC, telemetry logs)

Silver: Cleansed, deduplicated, standardized data

Gold: Business-level aggregates & reporting models

Scenario:

E-commerce pipeline:

- **Bronze:** raw order events
- **Silver:** validated orders with joins
- **Gold:** daily sales, customer metrics for dashboards

4. How Delta Lake handles ACID transactions?

Answer:

Delta Lake uses:

- Transaction log (`_delta_log`)
- Optimistic concurrency control
- Versioned data files

Scenario Example:

Two jobs updating same table won't corrupt data, because Delta ensures atomic commits and rollback if conflict occurs.

5. What are Z-Ordering and OPTIMIZE in Databricks?

Answer:

OPTIMIZE: compact small files → bigger Parquet files

Z-ORDER: sorts files on chosen column → faster filtering

```
OPTIMIZE sales ZORDER BY (customer_id)
```

Scenario:

Query filtering frequently on `customer_id` becomes faster by 40–70%.

6. Difference between Structured Streaming "Micro-batch" & "Continuous" mode?

Answer:

Mode	Latency	Use Case
Micro-batch (default)	~1 sec	ETL, dashboards
Continuous	~10–20 ms	High-frequency trading, IoT

Structured Streaming is recommended for most pipelines because it is stable and scalable.

7. What is Delta Live Tables (DLT) and when to use it?

Answer:

DLT automates:

- schema evolution
- data quality checks
- dependency management
- incremental processing

Example:

```
@dlt.table
def silver_orders():
    return dlt.read("bronze_orders").filter("status IS NOT NULL")
```

Scenario:

Used when building **production-grade ETL pipelines** with automatic retries and lineage tracking.

8. How do you handle Slowly Changing Dimensions (SCD Type 2) in Delta?

Answer:

Using **MERGE**:

```
MERGE INTO dim_customer t
USING updates u
ON t.customer_id = u.customer_id
WHEN MATCHED AND t.current = true AND u.hash <> t.hash THEN
    UPDATE SET t.current = false, t.end_date = current_date()
```

WHEN NOT MATCHED THEN

```
INSERT (customer_id, name, start_date, end_date, current)
VALUES (u.customer_id, u.name, current_date(), null, true)
```

Delta Lake makes SCD2 easy because MERGE is ACID compliant.

9. How do you tune Databricks jobs for performance?

Key Techniques:

- Use **OPTIMIZE + ZORDER**
- Use **Auto-Optimize + Auto-Compact**
- Reduce shuffle using:
 - `broadcast()`
 - `repartition()`
- Use **Photon engine**
- Avoid `spark.sql.shuffle.partitions` too high

Performance Tip:

```
spark.conf.set("spark.sql.shuffle.partitions", 200)
```

10. How does Unity Catalog help in governance?

Answer:

Unity Catalog provides:

- Centralized governance for all data assets
- Table-level, column-level, row-level security
- Lineage tracking
- Cross-cloud governance (AWS, Azure, GCP)

Scenario:

Finance department only accesses `masked_salary` field using column-level permissions.